

# Sky Finanzas — Infraestructura y Ciberseguridad

---

Documento de handover para el encargado de ciberseguridad. Describe la infraestructura actual de Sky Finanzas, los controles que la protegen, las brechas reconocidas y el plan de cierre. **Fuente de verdad doctrinal:** [Estados del Arte/SkyFinanzas\\_EstadoDelArte\\_v5\\_Documentado.pdf](#) (registrado ante INAPI, Chile). Este documento es derivado del v5; si algo aquí contradice al v5, gana el v5.

**Versión:** 1.0 · **Fecha:** 2026-05-08 · **Audiencia:** Ingeniero de seguridad incorporándose al equipo.

---

## 0. Cómo leer este documento

---

Sky vive simultáneamente en **dos backends**:

- **Backend Node.js** ( [backend/](#) ) — En **producción**, sirviendo usuarios reales en [api.skyfinanzas.com](#). Es la "Parte I" del v5.
- **Backend Python** ( [backend-python/](#) ) — Migración en curso, **no toca producción** hasta Fase 13. Es la "Parte II" del v5.

Cada control de seguridad se describe primero en su estado **actual (Node)**, después en su estado **objetivo (Python)**, y finalmente con la **deuda abierta** que separa uno de otro. Los identificadores [P0-x](#), [P1-x](#), [P2-x](#), [BUG-x](#) apuntan al inventario formal de la Parte III del v5.

---

## 1. Contexto y modelo de amenaza

---

### 1.1 Qué protege Sky

Sky almacena y procesa información **muy sensible** por usuario:

| Activo                                | Sensibilidad                | Dónde vive                                                                        |
|---------------------------------------|-----------------------------|-----------------------------------------------------------------------------------|
| Credenciales bancarias (RUT + clave)  | <b>Crítica</b>              | <code>bank_accounts.encrypted_rut</code> , <code>encrypted_pass</code> (cifradas) |
| Movimientos financieros               | Alta                        | <code>transactions</code> (Postgres, RLS)                                         |
| Saldos por cuenta                     | Alta                        | <code>bank_accounts.last_balance</code>                                           |
| Patrones de gasto                     | Media (anonimizada)         | schema <code>aria.*</code>                                                        |
| Identidad del usuario (UUID Supabase) | Media                       | <code>auth.users</code> (Supabase) y <code>profiles.id</code>                     |
| Conversaciones con Mr. Money          | Media (contexto financiero) | flujo a Anthropic, no persistido en DB                                            |

**Nota:** `profiles` no guarda nombre real, email, RUT ni documento de identidad — solo UUID y preferencias. La identificación PII se delega a `auth.users` administrado por Supabase.

## 1.2 Adversarios contemplados

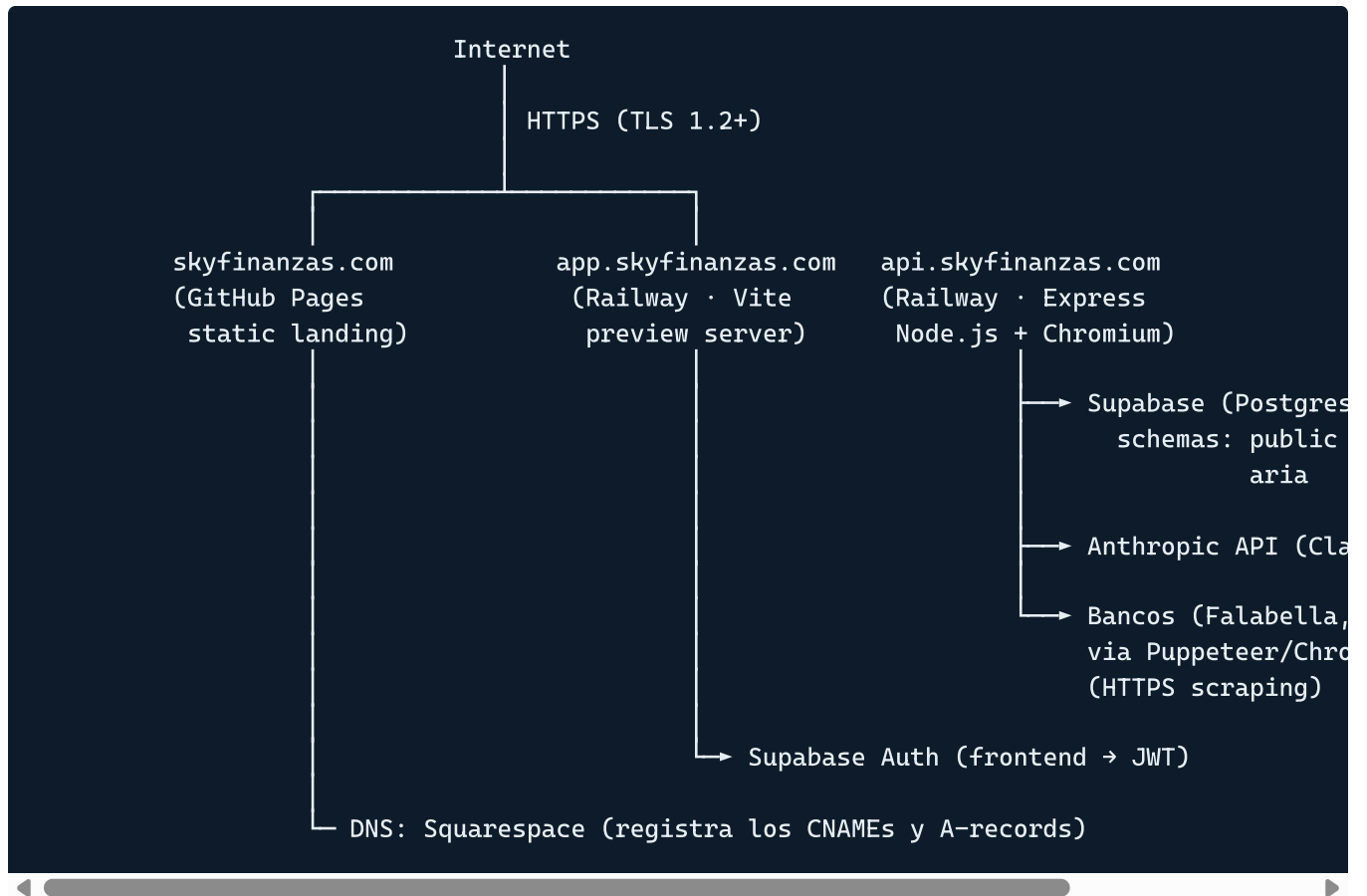
- **Atacante con acceso a la DB** (filtración Supabase, dump comprometido). Las credenciales bancarias deben quedar inservibles.
- **Atacante con UUID válido de un usuario** (UUIDs aparecen en logs, soporte, requests). No debe poder leer ni mutar datos de ese usuario.
- **Atacante con acceso al frontend o al browser** (XSS, devtools). No debe poder llamar a Anthropic, Supabase con `service_role` , ni ver claves del backend.
- **Atacante en la red** (MITM). Todo tránsito debe ser TLS.
- **Insider con acceso a logs**. Logs no deben contener passwords, RUTs, tokens.
- **Bots y abuso** disparando syncs en loop para quemar recursos. (Brecha abierta — P2-3.)
- **Caída de un proveedor bancario** (no es adversario humano pero es un riesgo de disponibilidad). Failover automático en arquitectura objetivo.

## 1.3 Adversarios fuera de scope (por ahora)

- Atacantes con acceso físico a la infraestructura de Railway o Supabase (delegamos en sus controles).
- Side-channel attacks en Chromium del scraper.
- Compromisos del SDK de Anthropic o Supabase publicados en npm/PyPI (mitigación parcial vía `package-lock.json` y futuras GitHub Actions con Dependabot — P2-2).

## 2. Topología de la infraestructura

### 2.1 Plano físico (servicios desplegados)



### 2.2 Plano lógico (regla de oro)

```
Frontend → solo muestra, captura input, llama al backend
Backend  → calcula, decide, guarda, llama a la IA
IA       → solo desde el backend, nunca desde el browser
ARIA     → solo escribe analytics anónimos
Cifrado  → solo el backend conoce BANK_ENCRYPTION_KEY
```

Esta regla está consagrada como **decisión doctrinal permanente** (v5 §26): cualquier diseño que la rompa debe ser rechazado en code review, no negociado.

## 2.3 Servicios externos invocados

| Servicio                           | Rol                                   | Auth                                       |
|------------------------------------|---------------------------------------|--------------------------------------------|
| Supabase (Postgres, Auth, Storage) | DB principal + IDP                    | JWT (anon) + service_role (server)         |
| Anthropic Claude                   | LLM Mr. Money + categorización capa 3 | API key ( <code>ANTHROPIC_API_KEY</code> ) |
| Banco Falabella, Banco de Chile    | Origen de movimientos                 | Scraping con RUT+clave del usuario         |
| Railway                            | PaaS (frontend + backend)             | OAuth (operadores)                         |
| Squarespace DNS                    | Resolución de dominios                | OAuth (operadores)                         |
| GitHub Pages                       | Landing pública                       | Repo público                               |
| Google Analytics                   | Analytics de landing                  | tag G-TQ06VZE8SF                           |

## 2.4 Procesos en producción

- `sky-frontend` (Railway): React 18.3 build con Vite, servido por `vite preview` . **No tiene acceso a secrets del backend**. Solo conoce `VITE_API_URL` y `VITE_SUPABASE_URL` + `anon` key.
- `sky-backend` (Railway, contenedor `node:22-slim` + Chromium vía `apt` ): Express 5.2. Único componente que conoce `BANK_ENCRYPTION_KEY` , `SUPABASE_SERVICE_KEY` , `ANTHROPIC_API_KEY` , `CRON_SECRET` .
- **Cron externo** (cron-job.org / EasyCron / GitHub Actions): hace `POST /api/internal/scheduled-sync` con header `x-cron-secret` . El backend valida el secret. Si la env `CRON_SECRET` no está definida, el endpoint responde **503 fail-safe**.

En arquitectura objetivo (Python) los procesos serán dos servicios deployables independientes: `sky-api` (FastAPI) y `sky-worker` (ARQ + Playwright), compartiendo Postgres y Redis. El API jamás importa Playwright.

## 3. Identidad y autenticación

### 3.1 IDP: Supabase Auth

- Sky **no** opera su propio sistema de cuentas. La identidad la maneja Supabase Auth.
- Métodos de login soportados: **email + password** y **Google OAuth**.
- Persistencia de sesión: handled por `supabase-js` en el frontend (localStorage, refresh tokens automáticos).
- Cada usuario se identifica por un **UUID** estable ( `auth.users.id` = `profiles.id` ).

## 3.2 Cliente Supabase: tres modos

Definidos en [backend/services/supabaseClient.js](#):

| Cliente                       | Key                               | Bypassa RLS | Uso                                                      |
|-------------------------------|-----------------------------------|-------------|----------------------------------------------------------|
| <code>getAnonClient()</code>  | <code>SUPABASE_ANON_KEY</code>    | No          | Operaciones del frontend (en backend casi no se usa hoy) |
| <code>getAdminClient()</code> | <code>SUPABASE_SERVICE_KEY</code> | Sí          | Uso operativo actual del backend                         |
| <code>getAriaClient()</code>  | <code>SUPABASE_SERVICE_KEY</code> | Sí          | Únicamente escribir en <code>aria.*</code>               |

`SUPABASE_SERVICE_KEY` **nunca** sale del backend. El frontend solo recibe la `anon` key (segura por diseño en Supabase).

## 3.3 Estado actual (Node) — P0-1 abierta

[backend/middleware/auth.js](#) lee `req.headers["x-user-id"]` y lo confía sin verificar:

```
export function extractUserId(req, res, next) {
  const userId = req.headers["x-user-id"] || null;
  req.userId = userId;
  next();
}
```

Combinado con `getAdminClient()` (que bypassa RLS), esto significa que **un atacante con un UUID válido** puede leer y mutar datos de ese usuario, **incluyendo disparar** `POST /api/banking/sync/:id` que descifra credenciales del banco en memoria.

**UUIDs no son secretos.** Aparecen en logs de Railway, en tickets de soporte, en requests del frontend. La mitigación actual es informal: "pocos conocen UUIDs ajenos". No es defensa criptográfica.

**Esto es la deuda P0-1 más urgente del producto.** Está documentada en v5 §11.2 y §20.1.

## 3.4 Estado objetivo (Python) — P0-1 cerrada

[backend-python/src/sky/api/middleware/jwt\\_auth.py](#) ya implementa la verificación correcta:

- Frontend envía `Authorization: Bearer <access_token>`.
- `jwt.decode(token, supabase_anon_key, algorithms=["HS256"], audience="authenticated")` valida firma, expiración y audience.
- Lanza `AuthenticationError` con `401` ante token ausente, expirado, audience incorrecta o firma inválida.
- `user_id = payload["sub"]` queda disponible vía `Depends(require_user_id)` en cada router.

Cuando se haga el cutover a Python (Fase 13), P0-1 queda cerrada por construcción. El frontend ya envía el `access_token` en la mayoría de los flujos — falta consolidarlo.

### 3.5 Endpoints internos (cron) — secret compartido

`backend/routes/internal.js` protege `/api/internal/*` con header `x-cron-secret`. Detalles importantes:

- Si `CRON_SECRET` **no está definida**, los endpoints responden **503**. Esto es **fail-safe**: nunca quedan abiertos por accidente.
- La comparación `provided !== expected` no es timing-safe — para un secret aleatorio de 32 bytes esto es aceptable, pero el equipo de seguridad debe migrar a `crypto.timingSafeEqual` cuando se toque ese archivo.
- En arquitectura objetivo, el endpoint vive en `sky.api.routers.internal` con la misma lógica + soporte para múltiples secrets rotables.

### 3.6 Recomendaciones inmediatas para el encargado de seguridad

1. **Cerrar P0-1 en Node antes de cualquier crecimiento de usuarios.** Hay dos caminos: (a) implementar `auth.getUser(token)` en `middleware/auth.js` y mover queries a clientes con JWT; o (b) acelerar el cutover a Python (Fase 13) que ya tiene la verificación correcta.
2. **Migrar la comparación de `CRON_SECRET` a `crypto.timingSafeEqual`** o equivalente.
3. **Establecer rotación periódica de `SUPABASE_SERVICE_KEY` y `BANK_ENCRYPTION_KEY`** (ver §6.5 abajo).

## 4. Capa de transporte y red

### 4.1 TLS

- **HTTPS end-to-end** en producción (Railway termina TLS, Squarespace DNS apunta los CNAMEs).
- Certificados gestionados por Railway (Let's Encrypt). El equipo no opera CSRs.
- HSTS: depende del default de Railway. **Acción de seguridad sugerida:** verificar y forzar HSTS con `Strict-Transport-Security` desde Express middleware.

### 4.2 CORS

`backend/server.js:27-57`:

```
const envOrigins = (process.env.CORS_ORIGINS || "")
  .split(",")
  .map(s => s.trim())
  .filter(Boolean);
```

Comportamiento actual:

- Hay allowlist explícita por env var `CORS_ORIGINS`.
- **Pero:** si `CORS_ORIGINS` está vacía, el backend **refleja cualquier origin** con un warning. Esto es la **deuda P1-2**.

- En producción esto es laxo: si alguien deploya sin setear la env, el origin se refleja.

En arquitectura objetivo (Python) ya está corregido: `backend-python/src/sky/api/main.py:88-92` **falla en arranque** si `NODE_ENV=production` y `CORS_ORIGINS` está vacío.

**Acción sugerida:** replicar este `fail-fast` en el Node `server.js` antes de cualquier nuevo deploy.

## 4.3 Headers permitidos

CORS actual permite `Content-Type`, `x-user-id`, `Authorization`, `x-cron-secret`. Cuando se cierre P0-1, `x-user-id` debe **eliminarse** del allowlist y del frontend para evitar regresiones silenciosas.

## 4.4 Rate limiting (deuda P2-3)

**No existe rate limiting hoy.** Un actor con un UUID válido (ver P0-1) puede disparar syncs bancarios en loop y quemar recursos del contenedor de Railway, además de potencialmente violar las cuotas de los bancos.

Plan documentado (v5 §21.3): - **Token bucket en Redis**, por usuario y por endpoint. - Endpoints sensibles (`/api/banking/*`, `/api/chat`) con límites estrictos. - También rate limiting **del lado del proveedor bancario** (no exceder cuotas del banco / Fintoc).

En arquitectura objetivo, el `RateLimiter` de `backend-python/src/sky/ingestion/rate_limiter.py` ya cubre el lado proveedor (sliding window log atómico en Lua). El rate limit HTTP público (slowapi) está marcado `TODO(Fase11)` en `main.py`.

---

## 5. Autorización: RLS y separación de schemas

### 5.1 Row Level Security en `public`

**Todas** las tablas de schema `public` tienen RLS habilitado: - `profiles`, `transactions`, `bank_accounts`, `goals`, `challenge_states`, `earned_badges`, `merchant_categories`.

Las políticas (definidas en `SupabaseSQLQuerys/public/*.sql`, repo separado) restringen lectura/escritura a `auth.uid() = user_id`.

**Pero:** el backend Node usa hoy `service_role` en casi todos los endpoints (ver §3.3). Esto **bypassa RLS**. La defensa real depende del JWT del cliente — que hoy no se verifica (P0-1).

Cuando se cierre P0-1, el patrón será: - Operaciones por usuario: cliente con JWT → RLS aplica. - Operaciones globales (escribir `merchant_categories`, `aria.*`): cliente con `service_role`.

### 5.2 Schema `aria` — bloqueado a clientes

`aria.*` solo es accesible con `service_role`. RLS no le aplica directamente porque el schema entero está fuera del path de los clientes anon.

**Esto es invariante doctrinal:** el frontend nunca debe poder leer ni escribir `aria.*`. Si alguna vez aparece un import de `getAriaClient()` en el frontend, es un bug crítico.

### 5.3 Vistas analíticas con threshold

Vistas como `aria.v_motivation_by_cohort` y `aria.v_spending_by_segment` exigen **mínimo 10 registros** por bucket (k-anonymity informal). Esto evita reidentificación incluso con accesos legítimos a las vistas.

## 6. Cifrado de credenciales bancarias

Esta es la sección más crítica para el encargado de seguridad. Las credenciales bancarias en texto plano **no deben existir** en la DB ni en logs.

### 6.1 Algoritmo y formato

`backend/services/encryptionService.js` y `backend-python/src/sky/core/encryption.py` implementan:

| Parámetro           | Valor                                                                      |
|---------------------|----------------------------------------------------------------------------|
| Algoritmo           | <b>AES-256-GCM</b>                                                         |
| IV length           | 16 bytes (128 bits) random por campo                                       |
| Tag length          | 16 bytes (128 bits, máximo de GCM)                                         |
| Derivación de clave | <code>SHA-256(BANK_ENCRYPTION_KEY)</code> → 32 bytes                       |
| Formato almacenado  | <code>base64(iv) + ":" + base64(authTag) + ":" + base64(ciphertext)</code> |
| Compatibilidad      | <b>Binario-compatible Node ↔ Python</b>                                    |

GCM es un AEAD: provee confidencialidad **y** autenticación. El `authTag` detecta tampering: si el ciphertext fue alterado, `decrypt()` lanza un error y nunca devuelve plaintext corrupto.

### 6.2 IV único por campo

Cada llamada a `encrypt()` genera un IV nuevo con `crypto.randomBytes(16)` / `os.urandom(16)`. Reutilizar un IV con la misma clave en GCM es catastrófico (rompe la confidencialidad). El código actual está correcto.

**Acción sugerida:** auditar regularmente que ningún PR introduzca un IV determinístico, hardcodeado o derivado del plaintext.



## 6.3 Verificación al arranque

Tanto Node como Python ejecutan un round-trip al arrancar: - Node: `verifyEncryptionReady()` invocado desde `server.js:22`. - Python: `verify_encryption_ready()` se llama en startup del worker (ver Fase 4 closure plan).

Si el round-trip falla (clave malformada, env ausente, dependencia rota), el proceso loggea **✗** y, en Python, **falla en arranque**. En Node hoy solo loggea — **acción sugerida**: convertirlo a `process.exit(1)` para fail-fast estricto.

## 6.4 Ciclo de vida de credenciales

Flujo (de v5 §6.3, [backend/services/bankSyncService.js](#)):

1. Usuario ingresa RUT + clave en frontend.
2. Frontend llama `POST /api/banking/connect` (HTTPS).
3. Backend cifra **inmediatamente** con `encrypt()` ([backend/routes/banking.js:65-66](#)) — el plaintext **muere ahí**.
4. Ciphertext va a `bank_accounts.encrypted_rut` y `encrypted_pass`.
5. En cada sync: `decrypt()` en memoria → uso por el scraper → variable se descarta al terminar la función. **Nunca** se logea (logging filter explícito).
6. Disconnect: el endpoint sobrescribe `encrypted_rut/pass = "REMOVED"` ([backend/routes/banking.js:222-223](#)) y marca `status = "disconnected"`.

## 6.5 Rotación de la clave maestra (deuda P2-6)

**Hoy no hay procedimiento documentado.** Si `BANK_ENCRYPTION_KEY` se compromete, el plan de rotación es manual y reactivo.

Plan documentado (v5 §21.5) — a implementar en migración Python:

- **Key versioning**: cada token guarda qué `key_version` lo cifró. Estructura propuesta: `v2:base64(iv):base64(tag):base64(ct)`.
- Rotación = cifrar nuevos tokens con versión nueva, mantener las viejas para decrypt de tokens existentes.
- Migración de tokens existentes en background.
- Retiro de la clave vieja cuando todos los tokens estén re-cifrados.

**Acción crítica para el encargado de seguridad:** este es probablemente el primer proyecto a implementar después del onboarding. Sin rotación procedimentada, una filtración de la clave es un evento de extinción.

## 6.6 Almacenamiento de la clave en Railway

`BANK_ENCRYPTION_KEY` y otras secrets viven como **environment variables de Railway** (panel UI). El equipo de operaciones (cofundadores) tiene acceso. **Acción sugerida**: auditar permisos del panel de Railway y

considerar mover a un secret manager dedicado (Vault, AWS Secrets Manager) cuando la operación crezca.

## 7. Catálogo de secretos

### 7.1 Inventario completo

| Variable                          | Dónde                                          | Quién lee                             | Si se filtra...                                                            |
|-----------------------------------|------------------------------------------------|---------------------------------------|----------------------------------------------------------------------------|
| <code>BANK_ENCRYPTION_KEY</code>  | Railway env (backend)<br>+ Python env          | Solo backend                          | Todas las credenciales bancarias<br>quedan descifrables → reset masivo     |
| <code>SUPABASE_SERVICE_KEY</code> | Railway env (backend)                          | Solo backend                          | RLS bypassed → lectura/escritura<br>total de la DB                         |
| <code>SUPABASE_ANON_KEY</code>    | Railway env (backend<br>+ frontend build-time) | Backend y<br>frontend                 | Bajo riesgo (es pública por diseño<br>en Supabase)                         |
| <code>ANTHROPIC_API_KEY</code>    | Railway env (backend)                          | Solo backend                          | Costos y cuota agotados; potencial<br>abuso                                |
| <code>CRON_SECRET</code>          | Railway env (backend)<br>+ cron-job.org config | Backend +<br>servicio cron<br>externo | Disparo no autorizado de syncs<br>masivos                                  |
| <code>DATABASE_URL</code>         | Railway env (backend<br>Python)                | Solo backend                          | Acceso directo a Postgres con<br>credenciales del owner                    |
| <code>REDIS_URL</code>            | Railway env (backend<br>Python)                | Solo backend                          | Lectura/escritura de circuit breaker,<br>rate limit, ARQ                   |
| <code>FINTOC_SECRET_KEY</code>    | Railway env (backend<br>Python, futuro)        | Solo backend                          | Acceso a la API de Fintoc del<br>workspace                                 |
| <code>SENTRY_DSN</code>           | Railway env (backend<br>Python, opcional)      | Solo backend                          | Bajo riesgo (DSN no es secreto<br>fuerte; pero permite spam de<br>eventos) |

### 7.2 Reglas inviolables sobre secrets

1. **Ninguno** de los secrets de la columna "Solo backend" puede vivir en el frontend, en el repo, en logs ni en URLs.
2. `.env` está en `.gitignore`. El equipo nunca commitea `.env`. Hay `.env.example` con valores ficticios.
3. Si un secret aparece en un log, el log debe truncarse y **el secret debe rotarse**.
4. Filtrado activo de PII en logs: `_PII_PATTERNS` en `backend-python/src/sky/core/logging.py:14-17` y `_SCRUB_KEYS` en `backend-python/src/sky/core/sentry_utils.py:16-29`.

## 7.3 Variables expuestas al frontend

Solo las prefijadas con `VITE_*` llegan al bundle de Vite: - `VITE_API_URL` — URL del backend (no secreto). - `VITE_SUPABASE_URL` — URL del proyecto Supabase (no secreto). - `VITE_SUPABASE_ANON_KEY` — Anon key (no secreto por diseño).

Confirmar siempre que **ninguna** otra var con prefijo `VITE_*` se introduzca jamás en el repo.

---

## 8. Pipeline ARIA — anonimización de analytics

---

ARIA = **Anonymized Randomized Intelligence Architecture**. Es el dataset propietario de comportamiento financiero chileno que Sky construye, **sin posibilidad de reidentificación individual**.

### 8.1 Cinco pasos del pipeline

Documentado en v5 §8 y [backend/services/ariaService.js](#):

1. **Extracción** — evento real → señal estructurada.
2. **Categorización** — valor exacto → rango (monto → bucket, fecha → trimestre).
3. **Eliminación de identidad** — `user_id` UUID **descartado** antes de escribir en `aria.*`.
4. **Randomización intra-bucket** — el valor guardado es **random dentro del rango**, no el real.
5. **Ruptura de correlaciones** — jitter temporal  $\pm 36h$ , `batch_id` propio por registro.

### 8.2 Consentimiento y la deuda P0-2

ARIA solo debe activarse si `profiles.aria_consent = true`. El usuario puede revocar en cualquier momento, lo que dispara cumplimiento de Ley 19.628 (derecho al olvido) vía `profiles.deletion_requested_at`.

**Brecha conocida (P0-2):** - `trackSpendingEvent(profile, tx, userId = null)` evalúa el guard de consent **solo si recibe** `userId`. - En el flujo de sync bancario, `fireAriaSignals` llama **sin pasar** `userId` → guard saltado → ARIA escribe sin verificar consent. - Los datos terminan **anonimizados y sin UUID** en `aria.*` (la integridad de los pasos 3-5 se mantiene), pero la **escritura no fue consentida**, lo cual viola la doctrina y debilita el cumplimiento Ley 19.628.

**Plan de cierre (v5 §20.2):** hacer `userId` requerido en las tres funciones `track*`, guard estricto, propagar desde todos los call sites. Estimación: 30 minutos. **Cerrar antes del próximo release**. En arquitectura objetivo (Python `sky.domain.aria`) esto se codifica desde el inicio.

### 8.3 Threshold de vistas

Vistas analíticas ( `v_motivation_by_cohort`, `v_spending_by_segment` ) exigen  $\geq 10$  **registros** por bucket. Si un bucket tiene menos, la vista lo omite. K-anonymity informal con  $k=10$  — no es una garantía formal pero impide reidentificación trivial.

## 8.4 Acceso a `aria.*`

Solo `service_role`. Sky internamente nunca lee `aria.*` desde el backend operativo — son vistas para análisis (interno hoy, B2B en horizonte). Cualquier export futuro **debe** pasar por las vistas con threshold, nunca por las tablas crudas.

---

## 9. Capa de scraping bancario

---

### 9.1 Aislamiento del proceso

Hoy: Express y Puppeteer **comparten proceso** en Node. Un scraper colgado degrada todos los endpoints.

Objetivo (Python): API y worker son **dos procesos deployables independientes**, comparten Postgres y Redis. El API jamás importa Playwright. Esta separación es invariante doctrinal (v5 §13.2).

### 9.2 Sanitización de errores del scraper

Errores del scraper pueden contener: passwords (si el scraper los logea por bug), RUTs, stack traces con paths internos, timeouts con detalles del DOM bancario.

`backend/services/bankSyncService.js` usa `sanitizeError()` (revisar implementación) que elimina estos campos antes de exponer al usuario o guardar en `last_sync_error`.

### 9.3 Browser pool y race conditions

- **Hoy (Node):** lock en memoria del proceso ( `Set` ). No funciona con múltiples workers — **BUG-3** en v5.
- **Objetivo (Python):** `pg_try_advisory_lock` con key derivada de `SHA-256(bank_account_id)`. Coordinación distribuida vía Postgres.

Esto es importante de seguridad porque **dos syncs concurrentes de la misma cuenta** pueden duplicar movimientos, corromper `last_balance` y, en el peor caso, gatillar bloqueo del usuario en el banco por intentos repetidos.

### 9.4 2FA en Banco de Chile

El scraper soporta 2FA por app bancaria: marca `status = "Esperando aprobación"` en la cuenta, el frontend hace polling. Timeout default 120s. **No** se intenta bypasear ni capturar el código 2FA — se delega al usuario en su app bancaria. Esto es correcto y debe preservarse.

---

## 10. Resiliencia: circuit breaker, rate limit, failover

---

Estos son controles que también son de **seguridad**, porque protegen contra ataques de disponibilidad y contra que un proveedor caído cascadee a Sky.

## 10.1 Circuit breaker por proveedor (Python objetivo)

[backend-python/src/sky/ingestion/circuit\\_breaker.py](#):

- Estado en Redis ( `cb:<source_id>` ), tres estados: `closed` , `open` , `half-open` .
- Abre tras **5 fallos en 60s**, mantiene abierto **120s**, cierra tras **3 éxitos consecutivos** en half-open.
- Si está abierto, el `IngestionRouter` salta al siguiente proveedor de la cadena.

## 10.2 Rate limiter por proveedor (Python objetivo)

[backend-python/src/sky/ingestion/rate\\_limiter.py](#):

- Sliding window log atómico en Redis (Lua) — `rl:<source_id>` .
- Namespaces separados de circuit breaker (no colisionan).
- Configuración via env: `RATE_LIMIT_OVERRIDES="scraper.bchile=2/60,fintoc=30/60"` .
- Si se excede: respuesta del router es `skip` (probar siguiente proveedor), **no fail** — Auth errors sí son `fail` .

## 10.3 Política de failover

[backend-python/src/sky/ingestion/routing/router.py](#):

- **Sí hace failover** ante `RecoverableIngestionError` y `circuit OPEN` .
- **No hace failover** ante `AuthenticationError` — la credencial es el problema, todos los proveedores la rechazarían igual. Prevenir que credenciales mal cargadas castiguen 5 proveedores en cadena.
- Si toda la cadena falla: `AllSourcesFailedError` .

## 10.4 Reglas de routing en DB

`public.ingestion_routing_rules` (migración `001_routing_rules.sql`): cadenas editables sin redeploy. Permite **canary releases** seguros: activar BCI directo al 5% → 50% → 100% con métricas en cada paso.

Esto también es defensa en profundidad: si BCI directo se compromete, `UPDATE` a la tabla baja el porcentaje a 0 sin deploy.

---

# 11. Logging y observabilidad sin filtrar PII

## 11.1 structlog con filter (Python)

[backend-python/src/sky/core/logging.py:14-17](#): regex

`(password|clave|rut|secret|token|api_key|authorization)` (case-insensitive) sobre **claves** de cada log dict → si matchea, valor reemplazado por `***REDACTED***` .

Convención: nunca pasar el valor sensible directamente como mensaje. Usar siempre key=value (`logger.info("auth", token=tok)` se redacta; `logger.info(f"token={tok}")` **no** se redacta).

### 11.2 Sentry con `before_send` PII scrubbing

`backend-python/src/sky/core/sentry_utils.py`: pipeline de dos pasos antes de enviar a Sentry:

- 1. `_scrub` **recursivo**: walk del evento. Claves en `_SCRUB_KEYS` → `[REDACTED]`. Strings que matchean regex de tokens ( `sk-ant- ...`, `sk-proj- ...` ) o RUT chileno ( `12.345.678-9` ) → `[REDACTED]`. Profundidad cap 10.
- 2. `_event_contains_sensitive` **(post-scrub)**: serializa el evento ya scrubbed a JSON y aplica los regexes una vez más. Si todavía detecta PII (token usado como key de dict, datos truncados por depth), **descarta el evento entero** ( `return None` ).
- 3. **Fail-safe**: cualquier excepción en cualquier paso → descarta el evento. Es preferible perder telemetría que filtrar PII.

`SENTRY_DSN` opcional — vacío = Sentry deshabilitado en dev.

### 11.3 Métricas Prometheus

`backend-python/src/sky/core/metrics.py`: contadores e histogramas por `source_id`, latencia, queue depth, circuit breaker state. Endpoint `/metrics` expuesto por FastAPI sin auth — **acción sugerida**: en producción ponerlo detrás de network policy de Railway o requerir cron-secret.

### 11.4 Logs en Railway (Node actual)

Hoy todos los logs van a `stdout` y se ven en el dashboard de Railway. **No hay agregación, no hay alerting, no hay retención formal**. Esto es deuda P2-4. Plan: integrar Sentry y Grafana/PagerDuty en Fase 10 de la migración Python.

## 12. Lifecycle de datos del usuario y cumplimiento

### 12.1 Marco regulatorio relevante

| Marco                                                     | Aplicabilidad                          | Estado en Sky                                                                        |
|-----------------------------------------------------------|----------------------------------------|--------------------------------------------------------------------------------------|
| <b>Ley 19.628</b> (Protección de Datos Personales, Chile) | Aplica directamente                    | Derecho al olvido implementado en <code>profiles.deletion_requested_at</code>        |
| <b>Sistema de Finanzas Abiertas (SFA)</b> — CMF Chile     | Aplicará cuando esté plenamente activo | Arquitectura objetivo soporta <code>SFA</code> como <code>SourceKind</code> (v5 §17) |
| <b>ISO/IEC 27001</b>                                      | Referencia (no certificación)          | Usado como guía de gestión de seguridad                                              |
| <b>PCI-DSS</b>                                            | <b>No aplica</b>                       | Sky no procesa pagos con tarjeta                                                     |

## 12.2 Borrado de cuenta

Flujo: 1. Usuario solicita borrado vía Mr. Money o soporte. 2. Backend setea `profiles.deletion_requested_at = now()`. 3. Job programado (worker) ejecuta hard-delete después del periodo legal: - `DELETE` en `transactions`, `bank_accounts`, `goals`, `challenge_states`, `earned_badges` por `user_id`. - `DELETE` en `profiles` por `id`. - `auth.users` se borra vía Supabase admin API. - **aria.\* no requiere acción** porque ya no contiene UUID — los datos están desligados de la identidad por construcción.

**Acción sugerida:** auditar trimestralmente que el job de deletion realmente se ejecuta y deja el estado consistente.

## 12.3 Consentimiento

- ARIA: opt-in explícito ( `aria_consent` ). Actualmente activo en flujo de onboarding del frontend.
- Cookies de la landing: minimal (Google Analytics). El producto autenticado no setea cookies de tracking propias.

---

## 13. Inventario de deuda de seguridad abierta

Tabla cruzada del v5 §19 con el repo actual. **El encargado de seguridad debería tomar esta lista como su backlog inicial.**

| ID       | Ítem                                                             | Severidad              | Fase de cierre                                                   | Estado                                                                     |
|----------|------------------------------------------------------------------|------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------|
| P0-1     | JWT no verificado en Node ( <code>x-user-id</code> confiado)     | Bloqueante             | Cutover Python (Fase 13)                                         | Abierto en Node, <b>resuelto en Python</b> ( <a href="#">jwt_auth.py</a> ) |
| P0-2     | Consent ARIA inconsistente en flujo bancario                     | Bloqueante             | Fix Node 30 min + reforzado en Python Fase 8                     | Abierto                                                                    |
| ~~P0-3~~ | ~~Refresh en vivo post-sync~~                                    | —                      | —                                                                | ✅ Resuelto Abr-2026                                                        |
| P1-1     | <code>Sky.jsx</code> god-component (1.678 LOC)                   | Estructural            | Refactor frontend paralelo                                       | Abierto                                                                    |
| P1-2     | CORS permisivo por fallback en producción                        | Estructural            | Fix Node + Python ya correcto                                    | Abierto en Node                                                            |
| P2-1     | Sin tests automatizados (solo Python tiene cobertura unit)       | Higiene                | Fase 10 + parity suite                                           | Parcial                                                                    |
| P2-2     | Sin CI/CD (Railway redepoya en push sin validar)                 | Higiene                | Fase 10                                                          | Abierto                                                                    |
| P2-3     | Sin rate limiting HTTP público                                   | Higiene                | Fase 11 (slowapi)                                                | Abierto                                                                    |
| P2-4     | Sin monitoring de errores (solo stdout en Railway)               | Higiene                | Fase 10                                                          | Parcial — Sentry ya integrado en Python con scrubbing                      |
| P2-5     | Paralelismo Puppeteer sin límite                                 | Higiene                | Browser pool en Python                                           | Mitigado (sync secuencial)                                                 |
| P2-6     | Rotación <code>BANK_ENCRYPTION_KEY</code> sin procedimiento      | <b>Higiene crítica</b> | Key versioning en Python                                         | Abierto                                                                    |
| BUG-1    | <code>external_id</code> con dos implementaciones inconsistentes | Bug                    | <code>build_external_id</code> único en Python                   | Abierto                                                                    |
| BUG-2    | Upsert apunta a UNIQUE INDEX inexistente                         | Bug                    | Migration <code>002_indexes_and_constraints.sql</code> (Fase 12) | Abierto                                                                    |



| ID           | Ítem                                                    | Severidad | Fase de cierre                              | Estado  |
|--------------|---------------------------------------------------------|-----------|---------------------------------------------|---------|
| <b>BUG-3</b> | Lock en memoria del proceso (no escala con N workers)   | Bug       | <code>pg_try_advisory_lock</code> en Python | Abierto |
| <b>BUG-4</b> | Sync secuencial entre bancos del mismo usuario (~5 min) | Bug       | Browser pool paralelo (~90s)                | Abierto |

## Prioridades sugeridas para las primeras 4 semanas

- Semana 1:** Cerrar P0-2 (30 min). Migrar `CRON_SECRET` a `timingSafeEqual`. Auditar `.env` de Railway y rotar keys que tengan más de 12 meses.
- Semana 2:** Cerrar P1-2 (CORS fail-fast en Node). Implementar HSTS y headers de seguridad ( `helmet` en Express).
- Semana 3:** Diseñar e implementar **key versioning** para `BANK_ENCRYPTION_KEY` (P2-6). Documentar runbook de rotación de emergencia.
- Semana 4:** Definir y empezar a aplicar la **rate-limiting policy** del lado HTTP. Configurar Sentry en producción del Node con `before_send` equivalente al Python.

P0-1 es la deuda más grave pero su cierre depende de coordinación con frontend (cambiar `x-user-id` por `Authorization`) y del calendario de cutover a Python — no es un fix de seguridad puro, requiere decisión de producto.

## 14. Escenarios de amenaza y mitigaciones

### 14.1 "Filtración de la base de datos Supabase"

- Atacante obtiene dump completo de Postgres.
- Movimientos:** legibles (no están cifrados a nivel de campo).
- Credenciales bancarias:** ciphertext. **Inservibles** sin `BANK_ENCRYPTION_KEY` (que vive en Railway, no en Supabase).
- Datos** `aria.*`: sin UUID, randomizados intra-bucket → no reidentificables.
- Acción inmediata:** rotar `SUPABASE_SERVICE_KEY`, forzar logout de todos los usuarios, comunicar el incidente, evaluar exposición de movimientos según fechas comprometidas.

### 14.2 "Filtración de `BANK_ENCRYPTION_KEY`"

- Worst case: atacante puede descifrar todas las credenciales bancarias.
- Mitigación inmediata:** invalidar todas las sesiones bancarias (forzar reconexión), **borrar** `encrypted_rut/pass` de todos los `bank_accounts`, comunicar incidente, rotar la clave.
- Mitigación estructural pendiente (P2-6):** key versioning permite rotación menos disruptiva.

### 14.3 "JWT robado / replay"

- Sesiones de Supabase tienen refresh tokens y expiración. Tokens robados son válidos hasta su `exp`.
- **Mitigación parcial actual:** los access tokens son cortos (1h por default Supabase).
- **Mitigación pendiente:** P0-1 (verificación real). Hoy el atacante ni siquiera necesita el JWT — basta el UUID en el header.

### 14.4 "Compromise del SDK de Anthropic / supabase-js"

- Supply chain attack vía npm/PyPI.
- **Mitigación parcial:** `package-lock.json` y `pyproject.toml` con versiones pinneadas.
- **Mitigación pendiente:** Dependabot + auditoría automática (P2-2). Política de no auto-update en producción.

### 14.5 "Phishing simulando Sky para robar RUT+clave"

- Riesgo de plataforma, no de infraestructura. Usuario engañado en sitio falso.
- **Mitigación:** dominio único `app.skyfinanzas.com`, comunicación clara en producto sobre nunca pedir clave fuera de la app, futura integración OAuth/SFA elimina la necesidad de password.

### 14.6 "Banco bloquea la cuenta del usuario por scraping"

- No es ataque a Sky pero es daño al usuario.
- **Mitigación:** rate limit por proveedor (en Python objetivo), retry policies con backoff exponencial (Fase 9), failover a agregador/API directa cuando esté disponible.

---

## 15. Reglas inviolables (resumen para code review)

---

Estas reglas se aplican en **toda** revisión de código tocando seguridad. Vienen del v5 §26 + §13.2 + §15.4 + §11.1.

1. `SUPABASE_SERVICE_KEY` y `BANK_ENCRYPTION_KEY` solo en backend. **Nunca** en frontend, repo, ni logs.
2. Credenciales bancarias = AES-256-GCM con IV único por campo (formato `iv:authTag:ciphertext` base64).
3. Mr. Money llama a Anthropic **solo desde el backend**. Nunca desde el browser.
4. RLS habilitado en **todas** las tablas de `public`. Schema `aria` bloqueado a clientes (solo `service_role`).
5. Frontend **nunca** llama a Supabase con `service_role` ni a Anthropic directo.
6. Errores del scraper sanitizados antes de mostrarse al usuario o guardarse en DB.
7. ARIA solo se activa con `aria_consent = true`. Sin UUID en `aria.*`. Service\_role exclusivo.
8. API Python **nunca** importa Playwright. El worker es el único con browser pool. (Aplicable post-cutover.)
9. `AuthenticationError` **no** dispara failover del IngestionRouter.
10. La deuda técnica se documenta, no se oculta.

Si un PR rompe alguna de estas reglas, **se rechaza sin negociación**. La doctrina sobrescribe conveniencia de corto plazo.

## 16. Runbook operativo del encargado de seguridad

### 16.1 Onboarding (primeros días)

- Leer este documento completo.
- Leer el v5 PDF ( `Estados del Arte/SkyFinanzas_EstadoDelArte_v5_Documentado.pdf` ), enfocándose en Partes I §11, II §13.2/§14.4/§15.4 y III completo.
- Acceso al panel de Railway (cofundadores aprueban), a Supabase, a GitHub, al cron externo.
- Auditar las env vars actuales y compararlas con `.env.example` de Node y Python.
- Hacer un round-trip de `verifyEncryptionReady` en local para confirmar el formato del key.

### 16.2 Tareas recurrentes

| Frecuencia  | Tarea                                                                                                         |
|-------------|---------------------------------------------------------------------------------------------------------------|
| Cada deploy | Revisar PRs por filtración de secrets, hardcoded keys, llamadas a <code>service_role</code> en frontend       |
| Semanal     | Revisar logs de Railway por errores que contengan strings sospechosos (RUT, "password", <code>Bearer</code> ) |
| Mensual     | Auditar dependencias ( <code>npm audit</code> , <code>pip-audit</code> )                                      |
| Mensual     | Verificar que el job de <code>deletion_requested_at</code> esté procesando                                    |
| Trimestral  | Rotar <code>CRON_SECRET</code> y <code>SUPABASE_SERVICE_KEY</code>                                            |
| Anual       | Rotar <code>BANK_ENCRYPTION_KEY</code> (cuando exista key versioning — P2-6)                                  |
| Anual       | Pentest externo (cuando el producto crezca)                                                                   |

### 16.3 Respuesta a incidentes

Niveles propuestos:

- **SEV-1**: filtración confirmada de `BANK_ENCRYPTION_KEY` o `SUPABASE_SERVICE_KEY` . Procedimiento: rotar inmediatamente, invalidar credenciales bancarias, notificar usuarios.
- **SEV-2**: brecha de auth (P0-1 explotada), abuso de un endpoint específico, scraping de un banco bloqueado masivamente. Procedimiento: parche en horas, comunicación interna.
- **SEV-3**: bug de seguridad sin explotación conocida. Procedimiento: ticket priorizado, fix en sprint normal.

Sin SLA formal hoy. **Acción sugerida**: definirlos junto con cofundadores en la primera semana.

## 16.4 Contactos clave

- **Cofundadores / titulares:** Cristian Vásquez (RUT 22.141.522-1), Juan José Latorre (RUT 22.003.365-1) — **SkyFinanzas SpA** RUT 78.395.382-K.
- **Soporte Supabase:** vía dashboard, plan de soporte según tier.
- **Soporte Railway:** vía dashboard.
- **Soporte Anthropic:** [support@anthropic.com](mailto:support@anthropic.com).

## 17. Glosario y archivos de referencia

### 17.1 Términos

| Término                  | Definición                                                                              |
|--------------------------|-----------------------------------------------------------------------------------------|
| <b>AEAD</b>              | Authenticated Encryption with Associated Data. AES-GCM lo provee.                       |
| <b>ARIA</b>              | Anonymized Randomized Intelligence Architecture. Pipeline propietario de anonimización. |
| <b>CanonicalMovement</b> | Modelo único de movimiento, devuelto por toda fuente. v5 §14.3.                         |
| <b>DataSource</b>        | Contrato Python para fuentes bancarias. v5 §14.                                         |
| <b>Mr. Money</b>         | Personaje IA de Sky. Guía, no decide.                                                   |
| <b>RLS</b>               | Row Level Security de Postgres.                                                         |
| <b>SFA</b>               | Sistema de Finanzas Abiertas chileno (Open Banking regulado por CMF).                   |

## 17.2 Archivos críticos para seguridad

| Archivo                                                                       | Propósito                                                |
|-------------------------------------------------------------------------------|----------------------------------------------------------|
| <a href="#">backend/services/encryptionService.js</a>                         | AES-256-GCM Node                                         |
| <a href="#">backend-python/src/sky/core/encryption.py</a>                     | AES-256-GCM Python (compat binario con Node)             |
| <a href="#">backend/middleware/auth.js</a>                                    | Auth Node (deuda P0-1)                                   |
| <a href="#">backend-python/src/sky/api/middleware/jwt_auth.py</a>             | JWT verification Python                                  |
| <a href="#">backend/services/supabaseClient.js</a>                            | Tres clientes Supabase con sus roles                     |
| <a href="#">backend/routes/banking.js</a>                                     | Endpoint de connect/sync — punto crítico de credenciales |
| <a href="#">backend/routes/internal.js</a>                                    | Cron secret-protected                                    |
| <a href="#">backend/server.js</a>                                             | CORS + bootstrap                                         |
| <a href="#">backend-python/src/sky/api/main.py</a>                            | Bootstrap Python con CORS fail-fast                      |
| <a href="#">backend-python/src/sky/core/sentry_utils.py</a>                   | PII scrubbing antes de Sentry                            |
| <a href="#">backend-python/src/sky/core/logging.py</a>                        | structlog filter                                         |
| <a href="#">backend-python/src/sky/ingestion/circuit_breaker.py</a>           | Circuit breaker en Redis                                 |
| <a href="#">backend-python/src/sky/ingestion/rate_limiter.py</a>              | Rate limiter por proveedor                               |
| <a href="#">backend-python/src/sky/core/config.py</a>                         | Catálogo completo de env vars                            |
| <a href="#">backend-python/.env.example</a>                                   | Template Python                                          |
| <a href="#">CLAUDE.md</a>                                                     | Instrucciones de proyecto + reglas inviolables           |
| <a href="#">Estados del Arte/SkyFinanzas_EstadoDelArte_v5_Documentado.pdf</a> | <b>Fuente de verdad doctrinal</b> (registrada INAPI)     |

## 17.3 Documentos complementarios

- [backend-python/docs/MIGRATION\\_13\\_PHASES.md](#) — plan maestro técnico de la migración.
- [backend-python/docs/REMEDIATION\\_P0\\_P3.md](#) — deuda P0-P3 mapeada a fase de cierre.
- [SupabaseSQLQuerys/](#) (repo separado) — todos los DDL incluyendo políticas RLS.

## 18. Cierre

Sky Finanzas tiene una postura de seguridad **honestamente documentada**: los controles activos están listados, las brechas también. Esa transparencia es parte de la confianza que el producto promete al usuario y que esta documentación promete al ingeniero que la hereda.

Las decisiones doctrinales (§15) **no se negocian** durante construcción. Si una propuesta nueva entra en conflicto con ellas, gana la doctrina hasta que un cambio explícito al v5 lo modifique — y ese cambio requiere registro INAPI.

El siguiente hito de seguridad es el **cutover a Python (Fase 13 del plan de migración)**, que cierra estructuralmente P0-1, P1-2, BUG-3, BUG-4 y prepara el terreno para los Fase 10/11 (monitoring, rate limit, tests, CI). Hasta entonces, las prioridades son las de §13.

*"La deuda técnica se documenta, no se oculta. Un registro honesto del estado real es parte de la confianza que el producto promete al usuario." — v5 §26.*